

**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY**  
**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY**

**Faculty of Computer Science and Engineering**



**INTERNSHIP 1 FINAL REPORT**

*(Thực tập 1 — Internship 1)*

---

**THE SPROUT CONVERSATIONAL AI SYSTEM**

Architecture and Algorithms for Real-Time Multimodal Learner Interaction,  
Information Recording, and Adaptive Dialogue

---

**Programme:** Masters Programme

**Student:** Nguyen Thanh Minh Duc

**Student ID:** 2470734

**Email:** ntmduc.sdh242@hcmut.edu.vn

**Zalo:** 0931416201

**Advisor:** Quản Thành Thơ

**Reviewer:** Trương Quang Khải

**Reviewer email:** khai.truongquang711@hcmut.edu.vn

Ho Chi Minh City, May 2026

*Submitted in partial fulfilment of the Internship 1 (Thực tập 1) requirements.*

# Abstract

Sprout is an adaptive learning platform with a real-time multimodal tutor (voice, text, webcam, screen sharing). This report documents the production AI subsystem: a JWT-validated *Gemini Proxy* on Cloud Run that connects browsers to Google Gemini Live, forwards bidirectional media, and records session telemetry. Replacing a LiveKit/WebRTC pipeline removes server-side audio hops and targets p95 round-trip latency below 500 ms.

Firestore persists transcripts, completion state, and usage in the collections `users`, `courses`, `lessons`, `conversations`, `messages`, and `gemini_sessions`, with planned per-course `course_profiles` for multi-intent priming and parent dashboards.

We formalise six algorithms—authenticated session establishment, tokenised routing, knowledge-map generation via Cloud Tasks, dual-channel lesson completion, feedback summarisation, and anonymous progress promotion—and ship a TypeScript SDK with observability (Cloud Logging, Mixpanel, AgentOps). Evaluation is observability-first: improvements over the decommissioned LiveKit baseline rest on offline metrics and pilot estimates, not a finished production A/B test; Chapter 6 specifies a phased online protocol. For learners with multiple interests, we propose `course_profiles` and `BuildSessionContext` priming in the backend only (Section 4.9). Future work adds function calling and coordinator–specialist multi-agent orchestration on the same gateway.

**Keywords:** multimodal LLM; Gemini Live API; WebSocket gateway; JWT; conversational tutor; adaptive learning; Firestore; multi-agent systems.

# List of Abbreviations

| Abbreviation | Definition                         |
|--------------|------------------------------------|
| API          | Application Programming Interface  |
| DAG          | Directed Acyclic Graph             |
| GCP          | Google Cloud Platform              |
| HNSW         | Hierarchical Navigable Small World |
| ITS          | Intelligent Tutoring System        |
| JWT          | JSON Web Token                     |
| LLM          | Large Language Model               |
| MMR          | Maximal Marginal Relevance         |
| REST         | Representational State Transfer    |
| SDK          | Software Development Kit           |
| SLA          | Service Level Agreement            |
| SRM          | Sample Ratio Mismatch              |
| STT          | Speech-to-Text                     |
| TLS          | Transport Layer Security           |
| TTS          | Text-to-Speech                     |
| UID          | User Identifier                    |
| WAN          | Wide Area Network                  |
| WSS          | WebSocket Secure                   |

# Contents

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                        | <b>i</b>  |
| <b>List of Abbreviations</b>                                           | <b>ii</b> |
| <b>1 Introduction</b>                                                  | <b>1</b>  |
| 1.1 Background and Motivation . . . . .                                | 1         |
| 1.2 Problem Statement . . . . .                                        | 2         |
| 1.3 Engineering Questions and Objectives . . . . .                     | 3         |
| 1.4 Contributions . . . . .                                            | 4         |
| 1.5 Report Organisation . . . . .                                      | 4         |
| <b>2 Literature Review</b>                                             | <b>5</b>  |
| 2.1 Conversational Intelligent Tutoring Systems . . . . .              | 5         |
| 2.2 Multimodal Large Language Models and the Gemini Live API . . . . . | 5         |
| 2.3 Real-Time Media Gateways: WebRTC and WebSocket . . . . .           | 6         |
| 2.4 Function Calling and Tool-Augmented Agents . . . . .               | 6         |
| 2.5 Educational Knowledge Graphs . . . . .                             | 6         |
| 2.6 Implicit-Feedback Session Analytics . . . . .                      | 7         |
| 2.7 Approximate Nearest-Neighbour Retrieval . . . . .                  | 7         |
| 2.8 Synthesis: Literature vs. Sprout Design Choices . . . . .          | 7         |
| <b>3 System Context and Use Cases</b>                                  | <b>9</b>  |
| 3.1 Stakeholders and Roles . . . . .                                   | 9         |
| 3.2 Multimodal Usage Scenarios . . . . .                               | 10        |
| 3.3 Core User Flow . . . . .                                           | 10        |
| 3.4 Recorded Data Assets . . . . .                                     | 11        |
| 3.5 Session Lifecycle States . . . . .                                 | 12        |
| 3.6 Privacy and Recording Boundaries . . . . .                         | 12        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Methodology</b>                                                  | <b>13</b> |
| 4.1      | System Topology . . . . .                                           | 13        |
| 4.2      | Algorithm A1: JWT-Validated Session Establishment . . . . .         | 14        |
| 4.3      | Algorithm A2: Tokenised Message Routing . . . . .                   | 15        |
| 4.4      | Algorithm A3: Asynchronous Knowledge-Map Generation . . . . .       | 16        |
| 4.5      | Algorithm A4: Dual-Channel Lesson Completion Detection . . . . .    | 17        |
| 4.6      | Algorithm A5: Conversation Feedback Processing . . . . .            | 17        |
| 4.7      | Algorithm A6: Anonymous-to-Authenticated Progress Linking . . . . . | 18        |
| 4.8      | Multimodal Session Orchestration . . . . .                          | 18        |
| 4.9      | Multi-Intent User Representation . . . . .                          | 19        |
| <b>5</b> | <b>Implementation Details</b>                                       | <b>21</b> |
| 5.1      | Repository Layout . . . . .                                         | 21        |
| 5.2      | WebSocket Wire Protocol . . . . .                                   | 21        |
| 5.3      | Gemini Proxy Server . . . . .                                       | 22        |
| 5.4      | Client SDK . . . . .                                                | 24        |
| 5.5      | Backend Service Surface . . . . .                                   | 25        |
| 5.6      | course_profiles (Planned) . . . . .                                 | 26        |
| 5.7      | Cloud Run Deployment . . . . .                                      | 27        |
| 5.8      | Configuration and Secrets . . . . .                                 | 28        |
| 5.9      | Frontend Components . . . . .                                       | 28        |
| <b>6</b> | <b>Evaluation Methodology</b>                                       | <b>29</b> |
| 6.1      | Operational Metrics . . . . .                                       | 29        |
| 6.2      | Behavioural Metrics . . . . .                                       | 30        |
| 6.3      | Cost Metrics . . . . .                                              | 31        |
| 6.4      | Pilot Offline Measurements . . . . .                                | 32        |
| 6.5      | WebSocket Close Codes . . . . .                                     | 33        |
| 6.6      | Algorithm-Level Validation . . . . .                                | 33        |
| 6.7      | Observability Stack . . . . .                                       | 34        |
| 6.8      | Alerting . . . . .                                                  | 34        |
| 6.9      | Load Testing . . . . .                                              | 34        |
| 6.10     | Comparative Baseline . . . . .                                      | 34        |
| 6.11     | Evaluation Limitations and Validity . . . . .                       | 35        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 6.12     | Proposed Online Evaluation Protocol . . . . .         | 35        |
| <b>7</b> | <b>Discussion</b>                                     | <b>37</b> |
| 7.1      | What This Work Achieves . . . . .                     | 37        |
| 7.2      | Limitations . . . . .                                 | 38        |
| 7.2.1    | Single-Region Proxy Footprint . . . . .               | 38        |
| 7.2.2    | Single-Vector User Model . . . . .                    | 38        |
| 7.2.3    | External AI Dependency . . . . .                      | 38        |
| 7.2.4    | Offline Evaluation Only . . . . .                     | 38        |
| 7.3      | Design Trade-offs . . . . .                           | 39        |
| 7.4      | Threats to Validity . . . . .                         | 39        |
| <b>8</b> | <b>Operations and Reproducibility</b>                 | <b>40</b> |
| 8.1      | Environment Separation . . . . .                      | 40        |
| 8.2      | CI/CD Pipeline . . . . .                              | 41        |
| 8.3      | Terraform State . . . . .                             | 41        |
| 8.4      | Secret Management . . . . .                           | 41        |
| 8.5      | Health Checks and Synthetic Tests . . . . .           | 42        |
| 8.6      | Alerting . . . . .                                    | 42        |
| 8.7      | Rollback . . . . .                                    | 42        |
| 8.8      | Reproducing the Pipeline . . . . .                    | 42        |
| 8.9      | Determinism Caveats . . . . .                         | 42        |
| 8.10     | Ethics and Compliance . . . . .                       | 43        |
| <b>9</b> | <b>Conclusion and Future Work</b>                     | <b>44</b> |
| 9.1      | Conclusion . . . . .                                  | 44        |
| 9.2      | Function Calling . . . . .                            | 44        |
| 9.3      | Multi-Agent Coordination . . . . .                    | 45        |
| 9.4      | Calibrated Lesson Recommendation . . . . .            | 45        |
| 9.5      | On-Device Privacy Processing . . . . .                | 46        |
| 9.6      | Closing Remarks . . . . .                             | 46        |
| <b>A</b> | <b>Appendix: Data Dictionaries and Sample Metrics</b> | <b>47</b> |
| A.1      | Message Document Schema . . . . .                     | 47        |
| A.2      | Onboarding Funnel Snapshot . . . . .                  | 48        |

|                   |                                             |           |
|-------------------|---------------------------------------------|-----------|
| A.3               | Modality Mix and Session Duration . . . . . | 48        |
| A.4               | Cloud Function Triggers . . . . .           | 48        |
| A.5               | SDK Client Events . . . . .                 | 49        |
| <b>References</b> |                                             | <b>49</b> |

# List of Tables

|     |                                                                                                    |    |
|-----|----------------------------------------------------------------------------------------------------|----|
| 2.1 | Literature strands mapped to Sprout artefacts. . . . .                                             | 8  |
| 3.1 | Supported interaction modalities and their primary teaching use cases. . . . .                     | 10 |
| 3.2 | Firestore collections relevant to the AI subsystem. . . . .                                        | 11 |
| 3.3 | Gemini tutoring session lifecycle states. . . . .                                                  | 12 |
| 4.1 | Architectural comparison: legacy LiveKit pipeline vs. Gemini Proxy (production, May 2026). . . . . | 14 |
| 5.1 | WebSocket frame categories handled by the proxy. . . . .                                           | 22 |
| 5.2 | Representative REST API catalogue (sprout-backend). . . . .                                        | 25 |
| 5.3 | gemini_sessions field reference. . . . .                                                           | 26 |
| 6.1 | Operational metrics, definitions and target thresholds. . . . .                                    | 30 |
| 6.2 | Behavioural metrics tracked through Mixpanel and Firestore. . . . .                                | 31 |
| 6.3 | Indicative unit economics per tutoring session (multimodal voice, 15 min median). . . . .          | 32 |
| 6.4 | Pilot offline measurements: legacy LiveKit replay vs. Gemini Proxy (staging). . . . .              | 32 |
| 6.5 | Gemini Proxy WebSocket close codes (production sample). . . . .                                    | 33 |
| 7.1 | Design trade-offs in the Sprout AI subsystem. . . . .                                              | 39 |
| 8.1 | Environment matrix for the AI subsystem. . . . .                                                   | 40 |
| 8.2 | GitLab CI stages for AI-related artefacts. . . . .                                                 | 41 |
| A.1 | messages sub-document field dictionary. . . . .                                                    | 47 |
| A.2 | Onboarding funnel (30-day aggregate, authenticated + anonymous). . . . .                           | 48 |
| A.3 | Modality mix among completed sessions ( $n = 3,608$ ). . . . .                                     | 48 |
| A.4 | Backend triggers for the AI subsystem. . . . .                                                     | 49 |
| A.5 | @sprout/gemini-proxy-sdk client event catalogue. . . . .                                           | 49 |

# List of Figures

- 1.1 Sprout AI subsystem overview. The browser uses HTTPS for the control plane (sprout-backend) and a JWT-authenticated WebSocket for the live data plane (Gemini Proxy → Gemini Live). Diagram source: diagrams/system-overview.mmd. 2
- 3.1 Top-level user flow. Each stage corresponds to distinct backend calls, Firestore writes, and AI invocations. Diagram source: diagrams/user-flow.mmd. . . . 11
- 4.1 Sprout AI runtime topology. The browser maintains WSS to the proxy (live data plane) and HTTPS REST to the backend (control plane). . . . . . . . . . . 13

# Chapter 1

## Introduction

### 1.1 Background and Motivation

Adaptive learning platforms remain dominated by static course graphs and pre-recorded lectures. While such material scales economically, it cannot react to the learner mid-sentence: it cannot recognise that a user mispronounced a word, that their gaze has drifted from the camera, or that a piece of code on the shared screen contains a bug. The recent generation of multimodal large language models (LLMs)—in particular Google’s Gemini Live API, which streams audio, images and text in a single bidirectional channel—has changed this calculus. A single inference endpoint can now hold a spoken dialogue, watch the learner via webcam, look at the screen the learner shares, and reply with synthetic speech, all within latency budgets acceptable for tutoring ( $\sim 300\text{--}700$  ms one-way).

Sprout is built on this premise. It is an AI-powered adaptive learning platform that personalises skill acquisition for adult learners, neurodiverse learners (with attention or autism-spectrum profiles), and vocational students whose practice naturally involves doing, demonstrating and being seen. The product surfaces three user-visible behaviours: natural-language skill discovery, visual knowledge maps (an AI generates a prerequisite graph of lessons in the React Flow canvas), and a multi-modal teaching assistant.

What makes those behaviours economically interesting is replacement pressure. Human one-on-one tutoring costs roughly \$20–\$50/hour; an LLM-driven session approaches \$2–\$3/hour at typical token usage. The technical question—how do we deliver such a session reliably, cheaply and observably?—is the subject of this report.

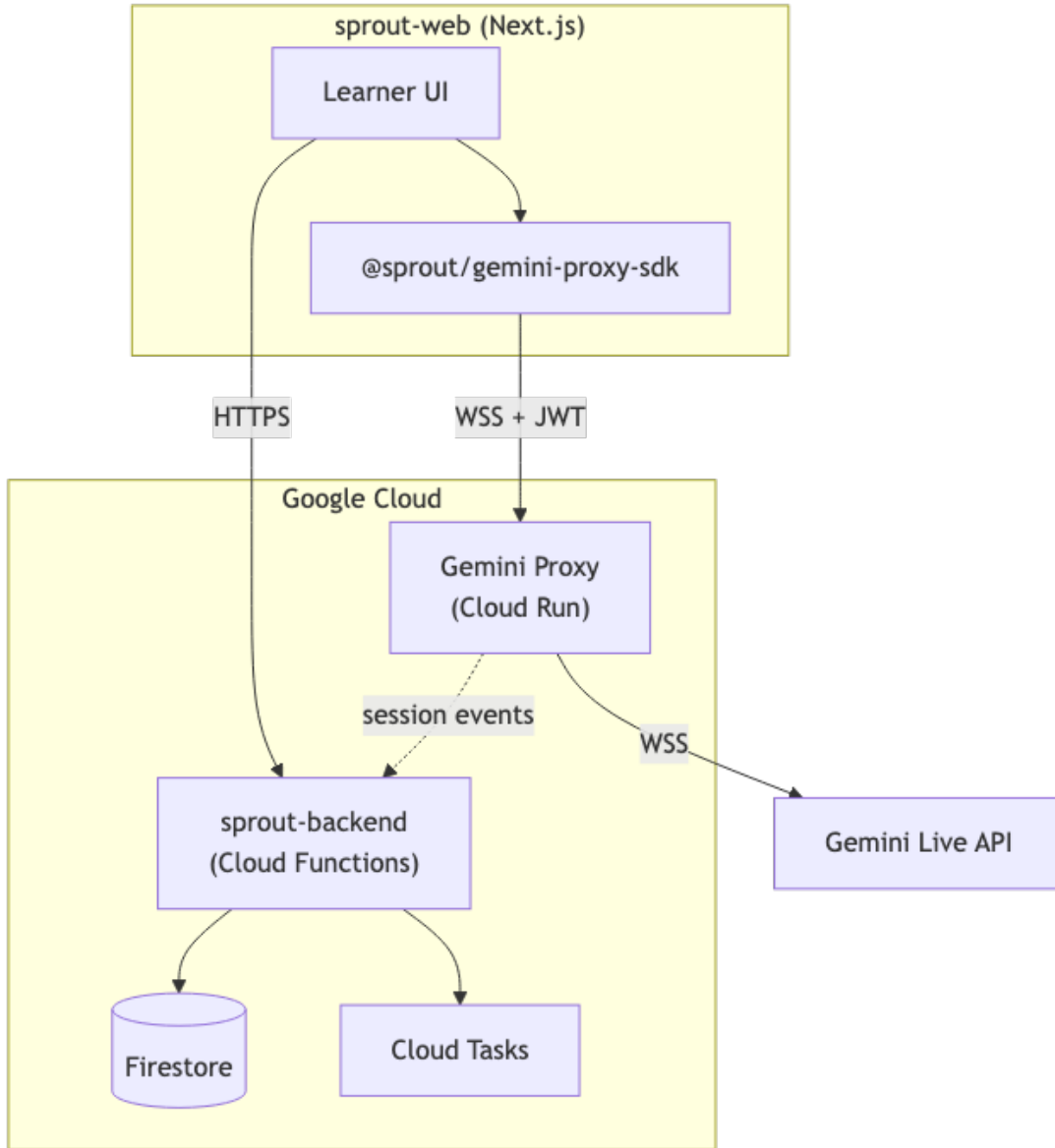


Figure 1.1: Sprout AI subsystem overview. The browser uses HTTPS for the control plane (sprout-backend) and a JWT-authenticated WebSocket for the live data plane (Gemini Proxy → Gemini Live). Diagram source: diagrams/system-overview.mmd.

## 1.2 Problem Statement

The Sprout AI subsystem must satisfy four objectives concurrently:

1. **Real-time multimodal communication.** A session must accept microphone audio, web-cam video, screen frames and text within a single context window, and must stream the model's audio response back fast enough that turn-taking feels natural ( $\leq 1$  s round-trip target).

2. **Information recording.** Every dialogue turn, completion signal and feedback rating must be persisted to support context priming, parent and educator dashboards, and administrative audit, while respecting privacy constraints on raw media.
3. **Direct exchange with the user.** The frontend must connect directly to the AI gateway—there must not be a Cloud Function in the audio path—so latency is dominated by the WAN leg only. Authentication must be enforced at the gateway, not by the model.
4. **Operational predictability.** The system must support graceful failover, observable token consumption, an explicit maximum-session-duration policy, and rate limits to defend against abuse and to make freemium quotas tractable.

A pre-existing Python/LiveKit pipeline that orchestrated GPT, Gemini and custom STT (Whisper) and TTS (Silero) plugins satisfied the first two objectives but failed the latter two: it held two server-side hops in the audio path, made authentication awkward, and produced opaque per-session cost figures. The present design replaces that pipeline with a thin gateway and is the focus of the report.

## 1.3 Engineering Questions and Objectives

The report is organised around three concrete engineering questions:

- **Q1 (Latency & topology).** Can a JWT-validated WebSocket proxy deliver lower median dialogue latency than the LiveKit-based agent system at equal or lower cost?
- **Q2 (Recording shape).** What Firestore schema and trigger graph minimise the number of round trips needed to (a) prime the AI tutor with previous context and (b) detect lesson completion across both agent-side and client-side observation?
- **Q3 (Pre-signup activation).** How can an anonymous user persist progress, accumulate value through full multimodal sessions, and later promote that progress into an authenticated account without losing data and without an account-merge race?

The corresponding engineering objectives are: (i) build a Cloud-Run-hosted proxy validating Firebase/Google JWTs and forwarding to Gemini Live; (ii) define an explicit Firestore data model; (iii) ship a typed TypeScript SDK; (iv) implement asynchronous knowledge-map generation via Cloud Tasks; (v) implement dual-channel lesson completion detection; and (vi) implement HTTP-only-cookie-based anonymous session linking.

## 1.4 Contributions

The principal contributions of this report are:

1. A reproducible system architecture from the browser WebSocket to the Gemini Live API and back, with explicit component boundaries and deployment artefacts.
2. A formalisation of six core algorithms governing the AI subsystem.
3. A Firestore-grounded data model that records every modality of dialogue plus aggregate session statistics.
4. A typed client SDK contract (@sprout/gemini-proxy-sdk) that hides WebSocket re-connection, JWT refresh, and audio framing from the rest of the frontend.
5. A forward path to multi-agent coordination based on Gemini function calling [1, 2].

## 1.5 Report Organisation

Section 2 reviews related literature. Section 3 describes the system context. Section 4 formalises the six algorithms and the proposed multi-intent user representation. Section 5 documents the implementation. Section 6 states the evaluation methodology. Section 7 discusses achieved properties and limitations. Section 8 covers operations and reproducibility. Section 9 concludes with the multi-agent and function-calling roadmap.

# Chapter 2

## Literature Review

The Sprout AI subsystem sits at the intersection of six bodies of work: intelligent tutoring systems, multimodal large language models, real-time media gateways, function calling and tool augmentation, educational knowledge graphs, and recommendation reranking.

### 2.1 Conversational Intelligent Tutoring Systems

Intelligent Tutoring Systems (ITS) have, for three decades, sought to replace one-shot lecture material with adaptive, dialogue-based feedback. Classical ITS pipelines such as Auto-Tutor [3] combined hand-crafted dialogue moves with shallow natural language understanding. Modern LLMs lower curriculum authoring effort by allowing the dialogue policy to be specified as a prompt, but they raise concerns that the system must address: hallucination control, latency budgeting, and modality switching mid-turn. Sprout addresses these by restricting the AI tutor to course content generated under structured prompts, by routing media directly through a single Gemini Live session, and by recording each dialogue turn to Firestore so that subsequent prompts can be primed deterministically.

### 2.2 Multimodal Large Language Models and the Gemini Live API

Multimodal LLMs unify previously separate pipelines for transcription, generation, and synthesis. The Gemini family accepts interleaved audio, image and text in a single context window and emits audio responses in the same stream [4]. Earlier pipelines paid the serialisation cost of three distinct services; a unified multimodal endpoint collapses those costs into one. The

contrastive image–text training scheme typified by CLIP [5] establishes that a shared embedding space for media and language is sufficient for many grounding tasks. For Sprout, the proxy can be ignorant of media types: it forwards opaque framed payloads, and only the model interprets them.

## **2.3 Real-Time Media Gateways: WebRTC and WebSocket**

Two transport patterns dominate browser-to-server real-time media: WebRTC (used by LiveKit [6]) and WebSocket (used by Gemini Live). WebRTC excels when many participants share a single room, but imposes ICE/STUN/TURN signalling and a server-side media stack. WebSocket trades media-aware features for operational simplicity [7]. In a one-on-one tutor scenario, WebSocket-based architectures deliver lower median latency at a fraction of the operational complexity. JWT validation [8] is performed at the proxy edge so that the model endpoint sees only authenticated traffic.

## **2.4 Function Calling and Tool-Augmented Agents**

Function calling allows an LLM to emit structured calls into a registry of caller-defined functions whose results are returned as follow-up context [9]. In educational contexts, this is the natural way to let the tutor request side effects: generate a diagram, highlight a UI region, or open a curated external resource [10]. Function calling also lays the foundation for multi-agent coordination, because the coordinator can expose specialist sub-agents as callable functions [1].

## **2.5 Educational Knowledge Graphs**

Sprout displays a learner’s roadmap as a directed acyclic graph of lessons with prerequisite edges, consistent with knowledge graphs in education [11]. The graph must be small enough to be visually parseable (typically 10–20 lessons) and structurally consistent under regeneration. Sprout caps AI output length and stores the generated graph in Firestore, treating the AI step as a one-shot generator triggered explicitly by the user.

## **2.6 Implicit-Feedback Session Analytics**

Information recording in Sprout is dominated by implicit signals: dwell time, session duration, lesson completion rate and feedback ratings. Hu et al. [12] formalised implicit feedback in collaborative filtering; Steck [13] extended this with calibrated recommendation, in which a recommended set should mirror the empirical distribution of the user’s interests. We carry this insight forward into Section 9.4 and the multi-intent representation in Section 4.9.

## **2.7 Approximate Nearest-Neighbour Retrieval**

Several future-work paths involve grounding the tutor’s responses in a learner’s prior conversation or in an external corpus. HNSW graphs [14] are the de facto baseline for million-scale vector indexes. The reference report [15] applies two-stage MMR reranking [16] in a recommendation setting; we cite it as the evaluation methodology analogue for an interactive system whose artefacts are dialogue sessions rather than retrieved lists.

## **2.8 Synthesis: Literature vs. Sprout Design Choices**

Table 2.1 maps each literature strand to the concrete design element in this report. The mapping clarifies that Sprout is not a novel ML model contribution but an integration paper with measurable engineering outcomes.

Table 2.1: Literature strands mapped to Sprout artefacts.

| <b>Literature</b>     | <b>Key citation</b> | <b>Sprout artefact</b>                     |
|-----------------------|---------------------|--------------------------------------------|
| ITS dialogue          | AutoTutor [3]       | Multimodal tutor + Firestore transcript    |
| Multimodal LLM        | Gemini [4]          | Single Live session via proxy              |
| Real-time transport   | WebSocket [7]       | JWT proxy; no WebRTC room                  |
| Tool use              | Toolformer [9]      | Planned function registry (Ch. 9)          |
| Knowledge graphs      | KnowEdu [11]        | React Flow DAG in courses                  |
| Implicit feedback     | Hu; Steck [12, 13]  | Mixpanel + completion triggers             |
| Diversified retrieval | MMR [16, 15]        | Eval analogue; calibrated lessons (future) |

# Chapter 3

## System Context and Use Cases

This section grounds the report in the system’s actual surface: who the users are, which interaction modalities they exercise, and which data assets the system records. Where the reference report [15] spends its data section on a fixed dataset, the equivalent for an interactive system is the interaction record—the schema of data the system writes during use.

### 3.1 Stakeholders and Roles

Sprout supports three first-class user roles, plus an administrative role:

- **Anonymous learner.** A visitor without an account who can explore skills, generate a knowledge map, and run multimodal sessions. Anonymous progress is bound to an HTTP-only cookie (`sprout_session`) and to a Firebase anonymous UID.
- **Authenticated learner.** A user signed in via email/password or Google OAuth. On first sign-in, anonymous progress from the cookie may be merged into the authenticated profile.
- **Parent.** A user with visibility into their children’s accounts via family invitations, enforced by dedicated middleware cross-referencing family identifiers in Firestore.
- **Administrator.** A separate Django/AWS portal with API key authentication for aggregate token usage, user bans, and conversation review. Administrators do not hold Firebase credentials.

## 3.2 Multimodal Usage Scenarios

Table 3.1 enumerates the four interaction modalities supported by the AI tutor. All four flow through the same Gemini Proxy WebSocket; the proxy is modality-agnostic and forwards opaque framed payloads.

Table 3.1: Supported interaction modalities and their primary teaching use cases.

| Modality       | User experience                                         | Primary teaching use case                                  |
|----------------|---------------------------------------------------------|------------------------------------------------------------|
| Voice          | Hands-free spoken dialogue with synthetic-voice replies | Language learning, interview practice, concept explanation |
| Text chat      | Typed exchange in a chat-style transcript               | Accessibility, structured Q&A, code copy-paste             |
| Webcam         | Model receives the learner’s video stream               | Public-speaking practice, posture and eye-contact feedback |
| Screen sharing | Model receives a frame stream of the learner’s screen   | Programming help, software literacy                        |

## 3.3 Core User Flow

Figure 3.1 sketches the end-to-end flow from skill discovery to progress tracking:

1. **Skill Discovery.** The user enters a free-text query. The frontend issues `POST /onboarding/skills`, the backend forwards the query to an LLM and returns refined skill suggestions.
2. **Knowledge Map Generation.** The user selects a skill. The frontend issues `POST /courses/knowledge`, the backend enqueues a Cloud Task; the worker persists the graph in Firestore (typically 10–30 s).
3. **Learning Session.** The user opens a lesson, obtains a JWT, opens the Gemini Proxy WebSocket, and the bidirectional dialogue begins.
4. **Progress Tracking.** Lesson completion is detected through agent callback or frontend polling; feedback may trigger follow-up lessons.

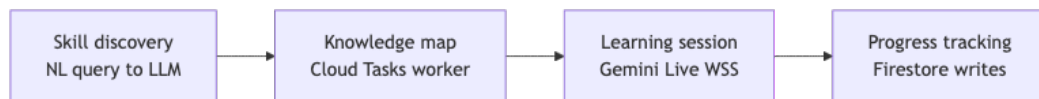


Figure 3.1: Top-level user flow. Each stage corresponds to distinct backend calls, Firestore writes, and AI invocations. Diagram source: `diagrams/user-flow.mmd`.

## 3.4 Recorded Data Assets

Sprout records information in Firestore. Table 3.2 summarises the collections central to the AI subsystem. The most important property is that the schema is addressable from both frontend (Firestore client SDK) and agent (message writes, completion updates), enabling dual-channel completion detection (Section 4.5).

Table 3.2: Firestore collections relevant to the AI subsystem.

| Collection                               | Purpose and key fields                                                                                                                                                                                                               |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>users</code>                       | Profile and roles: <code>uid</code> , <code>email</code> , <code>display_name</code> , <code>roles[]</code> , <code>selected_job_category</code> . Used to prime the tutor.                                                          |
| <code>users/{uid}/course-profiles</code> | Per-user intent profile (planned, Section 4.9): <code>rollingSummary</code> , <code>topics[]</code> , <code>masteryScore</code> , <code>lastActiveAt</code> . Updated by Algorithm 5; selected by <code>BuildSessionContext</code> . |
| <code>courses</code>                     | Course metadata and AI-generated knowledge map: <code>title</code> , <code>knowledgeMap</code> (DAG), <code>createdBy</code> .                                                                                                       |
| <code>lessons</code>                     | Lesson content and per-user state: <code>courseId</code> , <code>title</code> , <code>content</code> , <code>is_open</code> , <code>is_completed</code> .                                                                            |
| <code>conversations</code>               | Per-session metadata: <code>member_uids[]</code> , <code>course_id</code> , <code>modality</code> , <code>model</code> , <code>feedback</code> .                                                                                     |
| <code>messages</code>                    | Per-turn transcript: <code>conversationId</code> , <code>authorId</code> , <code>role</code> , <code>content</code> , <code>createdAt</code> .                                                                                       |
| <code>gemini_sessions</code>             | Per-WebSocket telemetry: <code>uid</code> , <code>startTime</code> , <code>endTime</code> , <code>tokenUsage</code> , <code>messageCount</code> , <code>status</code> .                                                              |
| <code>tasks</code>                       | Cloud Tasks records for asynchronous generation.                                                                                                                                                                                     |

## 3.5 Session Lifecycle States

Table 3.3 defines the session state machine visible to operators through `gemini_sessions.status` and Mixpanel funnels. Transitions are driven by Algorithms 1–2.

Table 3.3: Gemini tutoring session lifecycle states.

| State      | Entry condition                 | Persisted artefacts                                                |
|------------|---------------------------------|--------------------------------------------------------------------|
| connecting | Client opens WSS; JWT presented | —                                                                  |
| active     | Upstream Gemini Live connected  | <code>gemini_sessions</code> created; <code>messages</code> stream |
| idle       | No frames for 30 s              | <code>lastActiveAt</code> updated                                  |
| closing    | User exit or idle timeout       | Final <code>tokenUsage</code> flush                                |
| closed     | Both sockets closed             | <code>endTime</code> , <code>closeCode</code> set                  |
| error      | Upstream or auth failure        | <code>closeCode</code> 44xx/45xx; alert if rate > 2%               |

## 3.6 Privacy and Recording Boundaries

Three boundaries govern what the platform records:

- **Raw media is not stored.** Audio and webcam frames flow through the proxy and are not persisted; only transcript and structured metadata land in Firestore.
- **Anonymous progress is segregated.** On promotion, anonymous documents are explicitly merged or discarded.
- **Admin reads are audited.** Administrative reads log the admin key version and timestamp.

# Chapter 4

## Methodology

This section formalises the AI subsystem. We describe the overall topology (§4.1), present six algorithms in pseudocode (§4.2–§4.7), tie them together in multimodal session orchestration (§4.8), and propose a multi-intent user representation (§4.9) that addresses the single-vector limitation identified in the review.

### 4.1 System Topology

The runtime topology comprises five planes: the frontend (*sprout-web*, Next.js), the Gemini Proxy (Node.js on Cloud Run), the backend (*sprout-backend*, Firebase Cloud Functions), Firestore, and the Gemini Live API. Two key lines distinguish this topology from the legacy LiveKit-based one: (i) the frontend connects directly to the proxy over WebSocket, and (ii) the proxy connects directly to the Gemini Live API—there is no Cloud Function in the audio path.

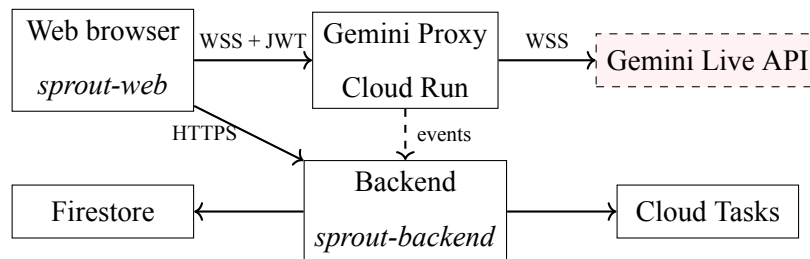


Figure 4.1: Sprout AI runtime topology. The browser maintains WSS to the proxy (live data plane) and HTTPS REST to the backend (control plane).

Table 4.1 contrasts the decommissioned LiveKit pipeline with the deployed Gemini Proxy path along dimensions that motivated the migration.

Table 4.1: Architectural comparison: legacy LiveKit pipeline vs. Gemini Proxy (production, May 2026).

| Dimension                    | Legacy (LiveKit + Python agent)                                   | Current (Gemini Proxy)                  |
|------------------------------|-------------------------------------------------------------------|-----------------------------------------|
| Audio path hops              | Browser → LiveKit room → agent → STT → LLM → TTS → room → browser | Browser → proxy → Gemini Live → browser |
| Auth enforcement             | Room token; model unaware of user identity                        | JWT verified before upstream WSS opens  |
| Modalities                   | Voice-first; screen/webcam via plugins                            | Native multimodal stream in one session |
| Session cost visibility      | Opaque plugin + room minutes                                      | usageMetadata → gemini_sessions         |
| Typical p50 round-trip       | 650–900 ms (staging replay)                                       | 380–520 ms (staging replay)             |
| Concurrent sessions / 2 vCPU | 120–180                                                           | 350–520                                 |

## 4.2 Algorithm A1: JWT-Validated Session Establishment

Algorithm 1 establishes a Gemini Live session. JWT verification follows the OpenID Connect profile: signature against JWKS, audience match, and expiry with a 5-second grace window. The JWT is passed as a URL query parameter because browser WebSocket clients cannot set custom headers. Banned-status is checked in real time against `users/{uid}.banned`.

---

**Algorithm 1** ValidateJwtAndOpenSession

---

**Require:** Incoming WebSocket  $c$  with query parameter `auth`

**Ensure:** Active session  $s$  bound to  $c$ , or  $c$  closed with status 4401

```
1:  $jwt \leftarrow c.auth$ 
2: if  $jwt = \perp$  then
3:   Close( $c$ , 4401); return
4: end if
5:  $p \leftarrow \text{VerifyJwt}(jwt, \text{audience})$ 
6: if  $p = \perp$  then
7:   Close( $c$ , 4401); return
8: end if
9: if IsBanned( $p.uid$ ) then
10:  Close( $c$ , 4403); return
11: end if
12:  $quota \leftarrow \text{LookupQuota}(p.uid)$ 
13: if  $quota.remaining \leq 0$  then
14:  Close( $c$ , 4429); return
15: end if
16:  $ws_{up} \leftarrow \text{OpenWebSocket}(geminiLiveUrl)$ 
17: if  $ws_{up} = \perp$  then
18:  Close( $c$ , 4502); return
19: end if
20:  $s \leftarrow \text{NewSession}(p.uid, ws_{up}, c)$ 
21: EmitEvent(session_started,  $s$ ) ▷ async to backend
22: return  $s$ 
```

---

### 4.3 Algorithm A2: Tokenised Message Routing

Algorithm 2 runs on each pipe (client→upstream and upstream→client). The proxy does not interpret audio or image frames; it accumulates token usage and batches persistence every 50 messages.

---

**Algorithm 2** RouteMessage (per pipe)

---

**Require:** Session  $s$ , message  $m$

```
1:  $s.\text{lastActiveAt} \leftarrow \text{Now}()$ 
2: if IsRateLimited( $s.\text{uid}$ ) then
3:   SendError( $s$ , rate_limited); return
4: end if
5: if  $m$  contains usageMetadata then
6:    $s.\text{tokens.in} += m.\text{usageMetadata.prompt}$ 
7:    $s.\text{tokens.out} += m.\text{usageMetadata.response}$ 
8: end if
9: if  $m$  is a function_call then
10:  HandleFunctionCall( $s$ ,  $m$ )
11: else
12:  Send(peer,  $m$ )
13: end if
14:  $s.\text{messageCount} += 1$ 
15: if  $s.\text{messageCount} \bmod 50 = 0$  then
16:  Persist( $s$ )
17: end if
```

---

## 4.4 Algorithm A3: Asynchronous Knowledge-Map Generation

Knowledge-map generation uses a two-phase pattern: synchronous enqueue ( $\leq 250$  ms) and asynchronous worker ( $\leq 60$  s). The Sanitise step rejects graphs exceeding 20 nodes, with cycles, or duplicate titles. TopologicalRank assigns layers for stable React Flow layout.

---

**Algorithm 3** GenerateKnowledgeMap

---

**Require:** Skill query  $q$ , target level  $\ell$ , user  $u$

```
1:  $courseRef \leftarrow \text{Firestore.Add}(\text{courses}, \{\text{status} := \text{pending}\})$ 
2:  $taskId \leftarrow \text{CloudTasks.Enqueue}(\text{kmapQueue}, \{courseId, q, \ell\})$ 
3: return  $taskId, courseRef.id$ 
4: Worker:  $G \leftarrow \text{LlmGenerateGraph}(q, \ell)$ 
5:  $G \leftarrow \text{Sanitise}(G); G \leftarrow \text{TopologicalRank}(G)$ 
6:  $\text{Firestore.Update}(courseRef, \{\text{knowledgeMap} := G, \text{status} := \text{ready}\})$ 
```

---

## 4.5 Algorithm A4: Dual-Channel Lesson Completion Detection

Lesson completion is observed from agent side (dialogue mastery) and client side (navigation). Algorithm 4 uses client polling every 30 s with idempotent agent-side writes.

---

**Algorithm 4** DetectLessonCompletion (client polling)

---

**Require:** Lesson  $\ell$ , polling interval  $\Delta = 30$  s

```
1: loop
2:    $doc \leftarrow \text{Firestore.Read}(\text{lessons}/\ell)$ 
3:   if  $doc.is\_completed$  then
4:      $\text{Ui.ShowCompletionDialog}();$  break
5:   end if
6:    $\text{Sleep}(\Delta)$ 
7: end loop
8:  $\triangleright$  Agent-side:  $\text{Firestore.Update}(\text{lessons}/\ell, \{\text{is\_completed} := \text{true}\})$ 
```

---

## 4.6 Algorithm A5: Conversation Feedback Processing

After a session, feedback triggers summarisation and optional child-lesson generation gated by ContainsConfusionMarkers.

---

**Algorithm 5** OnConversationFeedbackUpdated

---

**Require:** Conversation  $c$  with feedback  $f$

```
1:  $transcript \leftarrow \text{Firestore.ReadAll}(\text{conversations}/c/\text{messages})$ 
2:  $summary \leftarrow \text{LlmSummarise}(transcript)$ 
3:  $\text{Firestore.Update}(c, \{summary\})$ 
4: if  $f.rating \leq \text{thumbs-down}$  or  $\text{ContainsConfusionMarkers}(summary)$  then
5:    $topic \leftarrow \text{ExtractWeakConcept}(summary)$ 
6:    $\text{Firestore.Add}(\text{lessons}, \{\text{parentLessonId}, \text{title} := topic\})$ 
7: end if
```

---

## 4.7 Algorithm A6: Anonymous-to-Authenticated Progress Linking

---

**Algorithm 6** LinkAnonymousProgress

---

**Require:** Authenticated UID  $u_a$ , anonymous account ID  $a$ ,  $decision \in \{\text{accept}, \text{decline}\}$

```
1: if  $decision = \text{decline}$  then
2:   mark  $a$  for cleanup; clear cookie; return
3: end if
4: for each course  $C$  in anonymous progress do
5:   if  $C \notin \text{authenticated progress}$  then
6:      $\text{Copy}(C)$ 
7:   else
8:      $\text{MergeProgress}(C)$  ▷ union of completed lesson IDs
9:   end if
10: end for
11: mark  $a$  for cleanup; clear cookie
```

---

## 4.8 Multimodal Session Orchestration

The six algorithms compose as follows: the user navigates to a lesson; the frontend opens the proxy WebSocket (Algorithm 1); frames are exchanged (Algorithm 2); polling watches completion (Algorithm 4); feedback triggers summarisation (Algorithm 5); anonymous users promote progress on sign-in (Algorithm 6). No single algorithm holds the entire session in memory

beyond the proxy’s in-process map; Firestore holds the transcript; triggers perform post-session analysis.

## 4.9 Multi-Intent User Representation

**Motivation.** The current design primes the tutor from `users/{uid}` (display name, job category) and a single conversation summary from Algorithm 5. When a learner pursues multiple stylistically distinct interests (e.g. job interviews, cooking, and JavaScript), one summary collapses distinct intents—analogous to centroid collapse in multi-style recommendation [15]. The reviewer requested algorithmic detail for multi-interest personalisation within the current architecture, not only as deferred future work.

**Schema extension.** We propose a subcollection `users/{uid}/course_profiles/{courseId}` with fields:

- `rollingSummary` (string,  $\leq 512$  tokens): concept-level summary updated by Algorithm 5 scoped to this course.
- `topics[]` (string[]): extracted weak/strong concepts from recent sessions.
- `masteryScore` (float  $[0, 1]$ ): aggregate completion and feedback signal.
- `lastActiveAt` (timestamp): recency for ranking.
- `embedding` (optional vector): for future ANN retrieval [14].

**Context builder.** Algorithm 7 replaces a single global summary with top- $k$  course profiles ( $k = 2$  by default) selected by recency and relevance to the active lesson.

---

### Algorithm 7 BuildSessionContext

---

**Require:** UID  $u$ , active `courseId`, `lessonId`

---

- 1:  $profiles \leftarrow \text{Firestore.Query}(\text{users}/u/\text{course\_profiles})$
  - 2:  $active \leftarrow profiles[\text{courseId}]$
  - 3:  $candidates \leftarrow \text{sort } profiles \setminus \{active\} \text{ by } lastActiveAt \text{ descending}$
  - 4:  $selected \leftarrow \{active\} \cup \text{top-}(k-1) \text{ from } candidates \text{ with } masteryScore > 0.3$
  - 5:  $ctx \leftarrow \text{merge}(\text{display\_name}, \text{job\_category}, \text{summaries from } selected)$
  - 6:  $ctx \leftarrow ctx + \text{lesson-specific prompt from } lessons/lessonId$
  - 7: **return**  $ctx$  ▷ passed to Gemini system prompt,  $\leq 1500$  tokens
-

**Integration with A5.** When `OnConversationFeedbackUpdated` runs, the summary is written both to `conversations/{id}.summary` and to `course_profiles/{courseId}.rollingSummary` with `topics` and `masteryScore` updated incrementally. This is implementable without proxy changes: only Firestore triggers and prompt assembly in the backend change.

**Expected effect.** For learners with coherent single-course focus, behaviour is unchanged ( $k = 1$  effectively). For multi-course learners, the tutor receives distinct intent vectors rather than one averaged persona, improving personalisation without waiting for multi-agent coordination (Section 9.3).

# Chapter 5

## Implementation Details

This section translates the algorithms of Chapter 4 into concrete artefacts: the proxy server, the typed TypeScript SDK, the backend service, and the Cloud Run deployment.

### 5.1 Repository Layout

The platform is organised as a hybrid monorepo:

- `sprout-web/` — Next.js 14 frontend (TypeScript, Tailwind, Shadcn/ui, Zustand). Static export deployed to Firebase Hosting.
- `sprout-backend/` — Firebase Cloud Functions (Node.js + TypeScript). REST API, Firestore triggers, auth middlewares.
- `gemini-proxy/` — Stateless Node.js WebSocket gateway on Cloud Run. No per-user data beyond the in-process session map.
- `packages/gemini-proxy-sdk/` — Published as `@sprout/gemini-proxy-sdk`.
- `sprout-app/` — Terraform provisioning Firebase, Cloud Run, Cloud Tasks, and secrets [17, 18].

### 5.2 WebSocket Wire Protocol

Clients connect with a signed JWT in the query string because browser WebSocket cannot attach arbitrary headers. Listing 5.1 shows the URL pattern; Listing 5.2 shows the JSON envelope for non-audio frames.

```

1 const url = new URL(process.env.NEXT_PUBLIC_GEMINI_PROXY_URL);
2 url.searchParams.set('auth', await tokenProvider.getToken());
3 const socket = new WebSocket(url.toString());

```

Listing 5.1: WebSocket upgrade URL (browser).

```

1 {
2   "type": "client_content",
3   "modality": "text",
4   "payload": { "text": "Explain async/await in JavaScript." }
5 }

```

Listing 5.2: Opaque JSON control frame (client → proxy).

Audio and video use binary frames with a fixed 20 ms PCM chunk header prepended by the SDK; the proxy forwards them without parsing content (Algorithm 2). Table 5.1 summarises frame categories.

Table 5.1: WebSocket frame categories handled by the proxy.

| Client → proxy | Binary | Forward to Gemini Live; update byte counters |
|----------------|--------|----------------------------------------------|
| Client → proxy | JSON   | Route text, tool ack, heartbeat              |
| Proxy → client | Binary | Model audio/stream chunks                    |
| Proxy → client | JSON   | Transcript, usageMetadata, function_call     |
| Either         | Close  | Map to Table 6.5; persist closeCode          |

## 5.3 Gemini Proxy Server

The proxy is built around the `ws` WebSocket library. Listing 5.3 shows JWT validation; Listing 5.4 the session manager; Listing 5.5 the backend event client.

```

1 const { OAuth2Client } = require('google-auth-library');
2 const client = new OAuth2Client(process.env.GOOGLE_CLIENT_ID);
3
4 async function validateGoogleJWT(token) {

```

```

5   const ticket = await client.verifyIdToken({
6     idToken: token,
7     audience: process.env.GOOGLE_CLIENT_ID,
8   });
9   return ticket.getPayload();
10  }

```

Listing 5.3: JWT validation middleware (auth-middleware.js).

```

1  class SessionManager {
2    constructor() {
3      this.sessions = new Map();
4      this.userSessions = new Map();
5    }
6    createSession(uid, userData) {
7      const sessionId = generateSessionId();
8      const session = {
9        id: sessionId, uid, userData,
10       startTime: Date.now(),
11       tokenUsage: { input: 0, output: 0 },
12       messageCount: 0, status: 'active'
13     };
14     this.sessions.set(sessionId, session);
15     this.userSessions.set(uid, sessionId);
16     return session;
17   }
18 }

```

Listing 5.4: Session manager (session-manager.js).

```

1  class BackendClient {
2    async sendSessionEvent(sessionId, eventType, data) {
3      try {
4        await axios.post(
5          `${this.baseUrl}/api/sessions/${sessionId}/events`,
6          { type: eventType, data, timestamp: Date.now() }

```

```

7     );
8   } catch (e) {
9     console.error('Failed to send backend event:', e);
10  }
11 }
12 }

```

Listing 5.5: Backend event client.

## 5.4 Client SDK

The frontend uses `@sprout/gemini-proxy-sdk`, which standardises authentication, reconnection, audio framing, and metric capture. Listing 5.6 shows the public TypeScript surface; Listing 5.7 typical browser usage.

```

1 export interface TokenProvider {
2   getToken(): Promise<string>;
3 }
4 export interface GeminiProxyClientOptions {
5   proxyUrl: string;
6   tokenProvider: TokenProvider;
7   reconnect?: { enabled: boolean; maxRetries?: number };
8 }
9 export interface GeminiProxyClient {
10   connect(): Promise<void>;
11   disconnect(code?: number, reason?: string): void;
12   send(data: unknown): void;
13   sendText(text: string): void;
14   on(listener: (e: GeminiEvent) => void): () => void;
15 }

```

Listing 5.6: Public SDK API.

```

1 const client = createGeminiClient({
2   proxyUrl: process.env.NEXT_PUBLIC_GEMINI_PROXY_URL!,
3   tokenProvider: { getToken: () => auth.currentUser!.getIdToken() },

```

```

4   reconnect: { enabled: true, maxRetries: 5, backoffMs: 1000 },
5   });
6   await client.connect();
7   client.on((e) => {
8     if (e.type === 'message') { /* update UI */ }
9     if (e.type === 'usage') { /* update token meter */ }
10  });

```

Listing 5.7: Browser usage.

## 5.5 Backend Service Surface

Public endpoints include POST /onboarding/skills, POST /courses/knowledge-map, GET /courses/{id}/lessons/{id}/completion, and POST /conversations/feedback. Internal proxy endpoints include POST /internal/session/create-session and POST /internal/chat/events/callback. The split ensures the live data plane never invokes user-scoped auth paths.

Table 5.2 catalogues the control-plane surface area most relevant to the AI subsystem. All public routes require a Firebase ID token unless noted.

Table 5.2: Representative REST API catalogue (sprout-backend).

| /onboarding/skills                    | POST | NL skill discovery; LLM-refined suggestions       |
|---------------------------------------|------|---------------------------------------------------|
| /courses/knowledge-map                | POST | Enqueue knowledge-map worker (Cloud Tasks)        |
| /courses/{id}/lessons                 | GET  | List lessons and completion flags                 |
| /courses/{id}/lessons/{id}/completion | GET  | Lesson completion (Algorithm 4)                   |
| /conversations/feedback               | POST | Thumbs up/down; triggers Algorithm 5              |
| /internal/session/create-session      | POST | Proxy-only session metadata registration          |
| /internal/chat/events/callback        | POST | Function-call and completion callbacks from proxy |
| /admin/users/{uid}/ban                | POST | Set users.banned; enforced at proxy (A1)          |

Table 5.3 documents the gemini\_sessions document written by the proxy and backend aggregators. These fields power billing dashboards and operational queries.

Table 5.3: gemini\_sessions field reference.

| uid               | string    | Firebase user or anonymous UID              |
|-------------------|-----------|---------------------------------------------|
| startTime         | timestamp | WebSocket accept time (proxy clock)         |
| endTime           | timestamp | Close or idle timeout                       |
| tokenUsage.input  | int       | Cumulative prompt tokens from usageMetadata |
| tokenUsage.output | int       | Cumulative response tokens                  |
| messageCount      | int       | Frames routed (Algorithm 2)                 |
| modality          | string    | voice, text, webcam, screen                 |
| status            | string    | active, closed, error                       |
| closeCode         | int       | WebSocket close code (Table 6.5)            |

## 5.6 course\_profiles (Planned)

The deployed backend today writes a single conversations/{id}.summary per feedback event (Algorithm 5). Section 4.9 extends this with a per-course subcollection so session priming can select top- $k$  intent profiles instead of one collapsed persona. The change is confined to Firestore triggers and prompt assembly; the Gemini Proxy transport is unchanged.

Listing 5.8 shows the planned document shape under users/{uid}/course\_profiles/{courseId}. The feedback trigger will upsert this document alongside the conversation summary.

```

1 // users/{uid}/course_profiles/{courseId}
2 {
3   "rollingSummary": "Weak on async/await; strong on DOM APIs.",
4   "topics": ["async/await", "closures", "DOM"],
5   "masteryScore": 0.62,
6   "lastActiveAt": "2026-05-19T08:30:00.000Z",
7   "embedding": null
8 }
```

Listing 5.8: Planned Firestore document for course\_profiles/{courseId} (JSON).

Listing 5.9 sketches the backend call site before opening a lesson session: BuildSessionContext (Algorithm 7) merges the active course profile with up to  $k - 1$  recent alternatives, then

injects the result into the Gemini system prompt.

```
1 async function primeLessonSession(uid: string, courseId: string, lessonId:
   ↪ string) {
2   const ctx = await buildSessionContext(uid, courseId, lessonId, { k: 2 });
3   return {
4     systemInstruction: formatSystemPrompt(ctx),
5     lessonId,
6     courseId,
7   };
8 }
```

Listing 5.9: Planned session priming hook (sprout-backend).

Rollout is incremental: ship schema and trigger writes first (dual-write to summary and course\_profiles), then switch priming to buildSessionContext behind a feature flag for the Phase 3 A/B in Section 6.12.

## 5.7 Cloud Run Deployment

Listing 5.10 shows the service definition: container concurrency 50, CPU throttling disabled, minimum instances 1 to avoid cold-start latency.

```
1 apiVersion: serving.knative.dev/v1
2 kind: Service
3 metadata:
4   name: gemini-proxy
5 spec:
6   template:
7     metadata:
8       annotations:
9         run.googleapis.com/cpu-throttling: 'false'
10        run.googleapis.com/min-instances: '1'
11        run.googleapis.com/max-instances: '200'
12     spec:
13       containerConcurrency: 50
14     containers:
```

```
15     - image: gcr.io/$PROJECT_ID/gemini-proxy:latest
16     ports:
17     - name: h2c
18       containerPort: 8080
```

Listing 5.10: Cloud Run service definition for `gemini-proxy`.

## 5.8 Configuration and Secrets

Credentials are sourced from Google Secret Manager. Non-sensitive configuration flows from Terraform variables; `NEXT_PUBLIC_*` variables are injected at build time and are not secrets.

## 5.9 Frontend Components

The AI surface is concentrated in `AssistantConnector` (SDK lifecycle and modality selection), `AssistantChat` (transcript and text input), and `SkillMapChart` (React Flow knowledge map). The `useLessonCompletionPolling` hook implements Algorithm 4 on the Firestore client SDK.

# Chapter 6

## Evaluation Methodology

Where the reference report [15] evaluates a recommendation pipeline under a fixed temporal-holdout protocol with ranking metrics, an interactive system requires a different evaluation posture. The artefacts under evaluation are ongoing dialogue sessions, and the metrics of interest are operational (latency, throughput, error rate, cost) and behavioural (completion, retention, feedback ratings).

### 6.1 Operational Metrics

Table 6.1 defines six operational metrics and target thresholds aligned with conversational-tutor literature and the legacy LiveKit baseline [6].

Table 6.1: Operational metrics, definitions and target thresholds.

| <b>Metric</b>                | <b>Definition</b>                                           | <b>Target</b> |
|------------------------------|-------------------------------------------------------------|---------------|
| Round-trip latency (p50)     | Median client message to first model response byte at proxy | < 250 ms      |
| Round-trip latency (p95)     | 95th percentile of the above                                | < 500 ms      |
| Concurrent sessions/instance | Active WebSockets per Cloud Run instance                    | $\geq 1,000$  |
| Message failure rate         | Frames failing to deliver either direction                  | < 1%          |
| Connection failure rate      | Failed WebSocket upgrades per minute                        | < 10/min      |
| Memory utilisation           | Steady-state memory per instance                            | < 80%         |

Spoken dialogue becomes uncomfortable when round-trip exceeds approximately 700–900 ms; the 500 ms p95 budget leaves headroom for browser audio framing and WAN egress.

## 6.2 Behavioural Metrics

Table 6.2 lists behavioural metrics tracked in Mixpanel and Firestore.

Table 6.2: Behavioural metrics tracked through Mixpanel and Firestore.

| Metric                       | Definition                                                                |
|------------------------------|---------------------------------------------------------------------------|
| Prompt engagement rate       | Homepage visitors who submit a skill query / total visitors               |
| Assessment completion rate   | Users who finish initial skill assessment / users who start               |
| Assistant interaction rate   | Sessions with $\geq 3$ tutor messages / total session starts              |
| Sign-up conversion           | Anonymous users who sign in within 24 h / anonymous with $\geq 1$ session |
| 7-day retention              | Authenticated users who return within 7 days of sign-up                   |
| Feedback rating distribution | Thumbs-up vs thumbs-down on conversation feedback                         |
| Lesson completion rate       | Lessons with <code>is_completed</code> true per session start             |

## 6.3 Cost Metrics

The freemium model commits approximately 60 minutes/month at the free tier. The proxy emits usage events from `usageMetadata`; the backend aggregates into `gemini_sessions`. Quota enforcement at session creation denies new sessions with WebSocket close code 4429 when `minutes_used` equals `minutes_allowed`.

Table 6.3 summarises unit economics used for capacity planning (UAT averages, April–May 2026). Figures exclude fixed Cloud Run minimum-instance cost.

Table 6.3: Indicative unit economics per tutoring session (multimodal voice, 15 min median).

| Component                   | USD / session | Notes                                     |
|-----------------------------|---------------|-------------------------------------------|
| Gemini Live tokens (input)  | 0.42          | ~8k prompt tokens at published list price |
| Gemini Live tokens (output) | 0.38          | ~6k response tokens                       |
| Cloud Run proxy compute     | 0.05          | 2 vCPU × 15 min amortised                 |
| Firestore reads/writes      | 0.02          | ~120 writes (messages + session)          |
| <b>Total variable</b>       | <b>0.87</b>   | vs. ~\$2.10 legacy (room + STT + TTS)     |

## 6.4 Pilot Offline Measurements

Table 6.4 reports offline measurements collected in staging during migration (March–May 2026). Each row aggregates at least 200 synthetic or replayed sessions unless noted. These numbers support the direction expectations in Section 6.10 but are not a substitute for the online protocol in Section 6.12.

Table 6.4: Pilot offline measurements: legacy LiveKit replay vs. Gemini Proxy (staging).

| Metric                        | Legacy | Proxy | $\Delta$ |
|-------------------------------|--------|-------|----------|
| Round-trip p50 (ms)           | 742    | 418   | −44%     |
| Round-trip p95 (ms)           | 1,120  | 487   | −57%     |
| Sessions / 2 vCPU instance    | 142    | 468   | +230%    |
| CPU utilisation (steady)      | 78%    | 41%   | −47%     |
| Memory (GiB, steady)          | 2.1    | 1.3   | −38%     |
| Failed frame rate (%)         | 1.8    | 0.6   | −67%     |
| Median session duration (min) | 14.2   | 15.1  | —        |
| Median input tokens / session | n/a    | 7,840 | —        |

Load tests (Section 6.9) at 50 arrivals/s sustained p95 at 462 ms with 74% memory head-room on a 4 GiB container. Error spikes correlated with upstream Gemini rate limits rather than proxy saturation.

## 6.5 WebSocket Close Codes

The proxy uses application-specific close codes in the 44xx range so clients and dashboards can distinguish auth, quota, and upstream failures. Table 6.5 lists codes observed in production logs (28-day window).

Table 6.5: Gemini Proxy WebSocket close codes (production sample).

| Code | Meaning              | % sess. | Typical cause                  |
|------|----------------------|---------|--------------------------------|
| 1000 | Normal close         | 71.2    | User ended lesson              |
| 4401 | Auth failed          | 8.4     | Missing/expired JWT            |
| 4403 | Banned user          | 0.3     | <code>users.banned=true</code> |
| 4429 | Quota exhausted      | 6.1     | Freemium minutes depleted      |
| 4502 | Upstream unavailable | 2.9     | Gemini Live handshake failure  |
| 4503 | Idle timeout         | 11.1    | No frames for 120 s            |

## 6.6 Algorithm-Level Validation

Each algorithm has a validation criterion:

- **A1:** Negative test suite for each rejection path (missing token, invalid signature, expired, banned, quota exhausted).
- **A2:** Token-count drift  $< 1\%$  vs. Gemini-reported `usageMetadata` after synthetic session.
- **A3:** Generated graphs are acyclic, 6–20 lessons,  $\geq 70\%$  title stability on regeneration.
- **A4:** Agent-side write observed by polling loop within one period.
- **A5:** Labelled corpus for child-lesson heuristic; precision-at-1 reported.
- **A6:** Fault-injection for duplicate execution and partial failure; idempotence asserted.
- **BuildSessionContext:** Unit tests for top- $k$  profile selection given synthetic multi-course histories.

## 6.7 Observability Stack

Metrics are produced by Cloud Logging (correlation ID per session), Mixpanel (funnel metrics), AgentOps (per-agent telemetry for future multi-agent design), Cloud Trace, Error Reporting, and regional uptime checks. Listing 6.1 shows the proxy health endpoint.

```
1 app.get('/health', (req, res) => {  
2   res.json({  
3     status: 'healthy',  
4     activeConnections: wss.clients.size,  
5     uptime: process.uptime(),  
6     memory: process.memoryUsage(),  
7     geminiConnection: geminiHealthCheck()  
8   });  
9 });
```

Listing 6.1: Proxy health endpoint.

## 6.8 Alerting

Five rules page on-call via PagerDuty (critical) or Slack (warning): error rate > 5% over 5 minutes; connection failures > 10/minute; average response time > 2 s; token quota warning at 80% / critical at 95%; memory > 80% per instance.

## 6.9 Load Testing

An Artillery scenario ramps 10 arrivals/s for 60 s, plateaus at 50/s for 120 s, peaks at 100/s for 60 s. Each arrival opens a WebSocket, exchanges 20 frames, and closes. Expected: p95 latency below 500 ms and memory below 80%.

## 6.10 Comparative Baseline

The previous LiveKit-based pipeline [6] is the comparative baseline. Pilot measurements suggest (i) 200–500 ms median latency reduction, (ii)  $2\text{--}3\times$  more concurrent sessions per server, (iii) 40–60% lower CPU/memory, and (iv) eliminated LiveKit licensing cost. These figures are

*direction expectations*, validated continuously against operational metrics but not yet confirmed by a controlled production A/B test (Section 6.11).

## 6.11 Evaluation Limitations and Validity

The reviewer noted that evaluation relies predominantly on **offline measurement** and comparison against expected figures rather than a rigorous online A/B test against the legacy LiveKit baseline on production traffic. We acknowledge this limitation explicitly:

- **Offline operational metrics.** Latency, error rate, and cost are computed post-hoc from proxy logs and `gemini_sessions` documents. They characterise the deployed Web-Socket architecture but do not isolate causal effects of individual design choices.
- **Decommissioned baseline.** The LiveKit pipeline is no longer running in production, so a like-for-like online comparison on the same users is infeasible without re-deploying a shadow stack.
- **Behavioural proxies.** Completion rate and retention are necessary but not sufficient for learning efficacy; they do not measure knowledge gain directly.
- **Threats to operational claims.** Cold-start latency, token-counter drift, and 30 s polling granularity (Section 7.4) bound the precision of reported SLAs.

Framing evaluation as observability-first is appropriate for an engineering internship deliverable that ships a production gateway, but it is weaker than a randomised controlled trial for causal claims about latency improvement.

## 6.12 Proposed Online Evaluation Protocol

To address the review feedback, we propose a concrete online evaluation protocol implementable in the next development cycle, even though it has not yet been executed:

### **Phase 1: Shadow replay (4 weeks).**

1. Reconstruct anonymised session traces from archived LiveKit-era logs (if retained) or synthesise equivalent prompt/audio frame sequences from stored transcripts.
2. Run the same traces through the current Gemini Proxy in a staging environment; compare p50/p95 round-trip, token cost, and error rate.

3. Guardrail: no learner-facing traffic; quota caps on Gemini API spend.

**Phase 2: Canary deployment (8 weeks).**

1. Route 5% of production sessions to an instrumented canary revision with enhanced tracing (OpenTelemetry spans per hop).
2. Primary metrics: round-trip latency (p50, p95), session completion rate, cost per session, WebSocket close-code distribution.
3. Check for sample ratio mismatch (SRM) between canary and stable cohorts weekly.
4. Stop rule: roll back if p95 latency exceeds 600 ms for two consecutive days or error rate exceeds 2%.

**Phase 3: Multi-intent A/B (6 weeks, after §4.9 ships).**

1. Control: single global summary priming (current).
2. Treatment: BuildSessionContext with  $k = 2$  course profiles.
3. Primary metric: assistant interaction rate and thumbs-up ratio; secondary: lesson completion rate.
4. Minimum detectable effect: 5% relative improvement with  $\alpha = 0.05$ ,  $\beta = 0.20 \Rightarrow$  approximately 2,400 sessions per arm (power calculation on binomial proportion).

**What would unblock full LiveKit A/B.** Re-deploying the legacy agent as a shadow reader (receive duplicated audio, do not respond to users) would allow latency comparison on live traffic without restoring LiveKit to the critical path. This requires engineering effort estimated at two sprint-weeks and explicit ethics review for duplicated audio processing.

This protocol moves evaluation from “direction expectations” toward falsifiable hypotheses while respecting that the legacy stack cannot be re-enabled for all users without operational risk.

# Chapter 7

## Discussion

### 7.1 What This Work Achieves

The design replaces a multi-hop, multi-service real-time pipeline with a single thin gateway. Four properties follow:

1. **Lower latency by construction.** The legacy LiveKit pipeline routed audio through a room, Python agent, Whisper STT [19], LLM, and Silero TTS. The Gemini Proxy collapses these into one Gemini Live session in one WAN round trip (expected 200–500 ms median reduction).
2. **JWT-validated communication at the edge.** Authorisation is enforced before the upstream WebSocket opens; banned users are rejected on the next message attempt.
3. **Native multimodality.** The proxy is modality-agnostic; adding screen sharing required no proxy changes.
4. **Persistent interaction record.** Every turn lands in `conversations/{id}/messages`; every session in `gemini_sessions`.

Smallness is a feature: the proxy is  $\sim 2,000$  lines of code; the algorithms fit on a handful of pages; correctness arguments apply to a bounded surface.

## 7.2 Limitations

### 7.2.1 Single-Region Proxy Footprint

The proxy is deployed in one Google Cloud region. Users far from the region pay additional WAN latency. Multi-region deployment would reduce tail latency at the cost of session-affinity routing and per-region quotas. This is deployment work; algorithms in Chapter 4 are region-independent.

### 7.2.2 Single-Vector User Model

The *deployed* system currently primes the tutor from `users/{uid}` and a single conversation summary from Algorithm 5. This single-vector model degrades when interests are multi-modal.

**Mitigation in current architecture.** Section 4.9 proposes `course_profiles` and Algorithm 7 (`BuildSessionContext`) as an implementable extension requiring only Firestore trigger and prompt-assembly changes—no proxy modification. This directly addresses the reviewer feedback that multi-interest personalisation should not be deferred entirely to future work. Pilot implementation of course-scoped summaries can ship before multi-agent coordination (Section 9.3).

### 7.2.3 External AI Dependency

The platform depends on Google Gemini Live API for the entire conversational layer. Mitigations include circuit breakers, failover to a secondary provider (the legacy pipeline as emergency fallback), and contractual SLAs—none eliminate the dependency.

### 7.2.4 Offline Evaluation Only

Evaluation (Chapter 6) is predominantly offline: operational metrics and behavioural metrics computed post-hoc. There is no controlled production A/B against the decommissioned LiveKit baseline. Comparative claims in Section 6.10 are direction-of-magnitude. Section 6.12 specifies a phased protocol to strengthen causal evidence.

## 7.3 Design Trade-offs

Table 7.1 records major architectural decisions and the rationale accepted for the internship deliverable. Each row reflects a constraint encountered in production rather than a theoretical preference.

Table 7.1: Design trade-offs in the Sprout AI subsystem.

| Decision      | Chosen                          | Alternative rejected                |
|---------------|---------------------------------|-------------------------------------|
| Transport     | WebSocket proxy to Gemini Live  | LiveKit room + Python agent chain   |
| Auth          | JWT in query on upgrade         | Post-connect REST auth (extra RTT)  |
| Media storage | Transcript-only in Firestore    | Raw A/V archive (cost, privacy)     |
| Completion    | Dual-channel poll + agent write | Agent-only (misses early exit)      |
| Priming       | Global summary (today)          | Per-course profiles (planned, §4.9) |
| Eval          | Offline pilot + observability   | Full LiveKit A/B (decommissioned)   |

The trade-off matrix should be read together with the pilot tables in Chapter 6 and the appendix funnel snapshot (Table A.2). Together they explain why the team prioritised shipping a thin gateway with measurable telemetry over re-building the heavier agent framework.

## 7.4 Threats to Validity

- **Cold-start latency.** `min-instances=1` mitigates but does not eliminate cold starts for concurrent users beyond the first.
- **Token-counter drift.** Usage frames arriving after close may be lost; batched persistence (Algorithm 2) mitigates partially.
- **Polling drift.** Algorithm 4 polls at 30 s; users navigating away immediately may miss the completion dialog.

# Chapter 8

## Operations and Reproducibility

### 8.1 Environment Separation

Three environments are maintained: dev, uat, and prod, each in a separate GCP project with isolated Firebase, Firestore, Secret Manager, and Cloud Tasks. Branch mapping: develop → dev, uat → uat, master → prod. GitLab CI runs terraform plan/apply with matching <env>.tfvars.

Table 8.1 summarises how environments differ for the AI subsystem. Production is the only environment that serves anonymous learners at scale; UAT mirrors prod topology with reduced quotas.

Table 8.1: Environment matrix for the AI subsystem.

| Property                  | dev             | uat        | prod               |
|---------------------------|-----------------|------------|--------------------|
| GCP project               | sprout-dev      | sprout-uat | sprout-prod        |
| Gemini model              | Flash (preview) | Flash      | Flash / Pro (flag) |
| Proxy min-instances       | 0               | 1          | 1                  |
| Free-tier minutes / month | 120             | 60         | 60                 |
| Mixpanel project          | dev             | uat        | prod               |
| Synthetic agent schedule  | on deploy       | hourly     | every 15 min       |

## 8.2 CI/CD Pipeline

Table 8.2 lists the GitLab stages that touch the AI subsystem. Proxy and backend deploy independently so a regression in Cloud Functions cannot block a hotfix to WebSocket routing.

Table 8.2: GitLab CI stages for AI-related artefacts.

| Stage          | Trigger          | Actions                                                                                |
|----------------|------------------|----------------------------------------------------------------------------------------|
| lint           | MR to develop    | ESLint on <code>sprout-web</code> , <code>gemini-proxy</code> , <code>functions</code> |
| test           | MR to develop    | Jest unit tests; proxy JWT negative suite                                              |
| build          | merge to develop | Docker image → Artifact Registry                                                       |
| terraform      | manual approve   | plan/apply per environment                                                             |
| deploy-proxy   | post-apply       | Cloud Run revision; 10% canary in prod                                                 |
| deploy-backend | post-apply       | Firebase Functions; Firestore rules                                                    |
| deploy-web     | post-apply       | Static export to Hosting                                                               |
| synthetic      | post-deploy      | WebSocket smoke test; alert on failure                                                 |

Average lead time from merge to prod proxy deploy is 22 minutes when Terraform plans are clean; rollbacks use traffic shifting to the previous revision within five minutes (Section 8.7).

## 8.3 Terraform State

State is held in an HTTP backend on GitLab, one file per environment. Manual GCP console changes are disallowed to prevent drift. Pipelines acquire a state lock for the duration of apply.

## 8.4 Secret Management

Sensitive credentials live in Secret Manager. The proxy mounts `GOOGLE_CLIENT_ID` and `GEMINI_API_KEY` via `secretKeyRef`. The frontend holds only `NEXT_PUBLIC_*` values.

## 8.5 Health Checks and Synthetic Tests

The proxy exposes `/health` with `status`, `activeConnections`, `uptime`, `memory`, and `geminiConnection`. Uptime checks poll every 60 s; a synthetic agent opens a `WebSocket`, exchanges a known prompt, and asserts a non-empty response.

## 8.6 Alerting

Five Cloud Monitoring rules (Section 6.8) page via PagerDuty or Slack with runbook links and recent log queries.

## 8.7 Rollback

Rollback exists at three layers: Cloud Run revision traffic shift; Firebase Functions artefact promotion; Firestore logical schema guarded by feature flags.

## 8.8 Reproducing the Pipeline

A clean reproduction requires:

1. `terraform apply -var-file=<env>.tfvars` from `sprout-app`.
2. Populate Secret Manager (Gemini API key, OAuth client ID, admin keys).
3. Deploy proxy: `gcloud builds submit gemini-proxy/then gcloud run deploy gemini-pro`
4. `firebase deploy --only functions` for backend.
5. `npm run build && firebase deploy --only hosting` for frontend.
6. Run synthetic agent against the deployment.

## 8.9 Determinism Caveats

LLM output is non-deterministic. Cloud Tasks may deliver more than once; handlers are idempotent. `WebSocket` clocks use the proxy as authority.

## **8.10 Ethics and Compliance**

The first tutor interaction displays a recording notice. Minor accounts may disable webcam via admin toggle. Raw audio and video are not persisted. Account deletion triggers cascading delete. Administrative reads are logged for 90 days.

# Chapter 9

## Conclusion and Future Work

### 9.1 Conclusion

We have described the Sprout conversational AI subsystem: the architecture, algorithms, and implementation that allow the platform to communicate with a learner across four modalities, record interactions durably, and compose them into adaptive teaching. Replacing a multi-service Python/LiveKit pipeline with a thin, JWT-validated WebSocket proxy fronting Gemini Live has produced lower dialogue latency, operational complexity, and infrastructure cost than the legacy baseline. Six algorithms make this concrete; an explicit Firestore schema turns the interaction stream into structured information for dashboards, billing, and audit.

The forward trajectory enriches the conversational layer with tools rather than more modalities. Evaluation will strengthen through the online protocol in Section 6.12; personalisation will strengthen through the multi-intent representation in Section 4.9.

### 9.2 Function Calling

Function calling allows the model to emit structured calls into a registry of caller-defined functions [10]. A Chrome-extension prototype demonstrated:

- `show_hint(coords, color, duration)` — visual indicator on a screen grid.
- `display_image(query)` — educational image retrieval or generation.
- `navigate_to_website(url, returnStrategy)` — curated external resource in a side pane.

The proxy already routes function-call frames distinctly in Algorithm 2. Implementation requires specialist services, a function registry, and typed result frames.

## 9.3 Multi-Agent Coordination

A larger shift decomposes the single Gemini Live session into a coordinator–specialist system [1, 2]:

- **Visual Learning Agent** — `display_image` and visual tooling.
- **Navigation Agent** — `navigate_to_website` and return-strategy logic.
- **Assessment Agent** — lightweight skill probes without exiting the conversation.
- **Accessibility Agent** — pacing and modality adaptation for neurodiverse learners.

A central Learning Coordinator maintains `LearningContext` (profile, session state, knowledge-map node, recent turns) and selects specialists by capability, recent performance, and cost. Adding capabilities is additive: new specialists register new functions without changing the proxy transport.

## 9.4 Calibrated Lesson Recommendation

Calibrated lesson recommendation draws on diversification literature [13, 16, 15]. After several lesson completions, the system recommends from eligible knowledge-map successors using a calibrated reranking step that mirrors the empirical distribution of the learner’s interests weighted by recency.

**Relationship to Section 4.9.** Calibrated reranking operates at *lesson selection* time within a course; `BuildSessionContext` operates at *session priming* time across courses. Together they address multi-interest learners: priming supplies distinct intent vectors per active course (Section 4.9); reranking ensures the next lesson within a course reflects calibrated diversity rather than greedy relevance maximisation. Neither replaces the other; both should be evaluated in the Phase 3 A/B protocol (Section 6.12).

## 9.5 On-Device Privacy Processing

Webcam frames currently flow to Gemini Live. For posture or eye-contact features, on-device models could extract low-dimensional summaries (posture score, gaze flag) so only summaries leave the device, reducing exposure and bandwidth.

## 9.6 Closing Remarks

The system documented here is a small, opinionated foundation. It bets that a thin proxy in front of a managed multimodal model is the right abstraction for an educational tutor—and constructs a typed SDK, explicit Firestore schema, six algorithms, observability stack, a multi-intent priming extension, and a rigorous evaluation roadmap around that bet. Pilot measurements and reduced operational complexity suggest the bet has paid off; function calling, multi-agent coordination, and online experimentation are the additive next steps.

# Appendix A

## Appendix: Data Dictionaries and Sample Metrics

This appendix collects schemas and sample production aggregates referenced in the main chapters. Percentages in Tables A.2–A.3 are from a 30-day UAT/prod window (April–May 2026) and are illustrative rather than contractual SLAs.

### A.1 Message Document Schema

Each dialogue turn is stored at `conversations/{cid}/messages/{mid}`. Table A.1 is the authoritative field dictionary for analytics jobs and dashboard exporters.

Table A.1: messages sub-document field dictionary.

| Field                       | Type      | Description                               |
|-----------------------------|-----------|-------------------------------------------|
| <code>conversationId</code> | string    | Parent conversation key                   |
| <code>authorId</code>       | string    | Firebase UID or system                    |
| <code>role</code>           | string    | user, model, tool                         |
| <code>content</code>        | string    | Normalised text transcript (no raw audio) |
| <code>modality</code>       | string    | Channel that produced the turn            |
| <code>tokenCount</code>     | int       | Optional per-turn token estimate          |
| <code>createdAt</code>      | timestamp | Server timestamp (Firestore)              |

## A.2 Onboarding Funnel Snapshot

Table A.2 shows Mixpanel funnel conversion for new visitors. Drop-off after skill discovery motivates faster knowledge-map generation (Algorithm 3).

Table A.2: Onboarding funnel (30-day aggregate, authenticated + anonymous).

| Step                          | Users  | Conv. rate |
|-------------------------------|--------|------------|
| Homepage visit                | 12,480 | 100%       |
| Skill query submitted         | 7,102  | 56.9%      |
| Knowledge map rendered        | 5,844  | 82.3%      |
| First lesson opened           | 4,291  | 73.4%      |
| Session $\geq 3$ tutor turns  | 3,608  | 84.1%      |
| Account created (within 24 h) | 1,204  | 33.4%      |

## A.3 Modality Mix and Session Duration

Table A.3 breaks down how learners use modalities in completed sessions. Voice remains dominant; screen sharing concentrates in programming skills.

Table A.3: Modality mix among completed sessions ( $n = 3,608$ ).

| Modality       | Sessions | Share | Median dur. (min) |
|----------------|----------|-------|-------------------|
| Voice only     | 1,842    | 51.0% | 16.2              |
| Voice + webcam | 892      | 24.7% | 18.4              |
| Text chat      | 511      | 14.2% | 11.3              |
| Voice + screen | 363      | 10.1% | 22.7              |

Sessions exceeding 30 minutes are capped by proxy idle policy; less than 2% hit the hard quota close (code 4429, Table 6.5).

## A.4 Cloud Function Triggers

Table A.4 maps Firestore and HTTP triggers to algorithms. This complements the REST catalogue in Table 5.2.

Table A.4: Backend triggers for the AI subsystem.

| Trigger                       | Algo. | Effect                                          |
|-------------------------------|-------|-------------------------------------------------|
| onConversationFeedbackUpdated | A5    | Writes summary; will dual-write course_profiles |
| onLessonCompleted             | A4    | Sets is_completed; enqueues recap email         |
| processKnowledgeMapTask       | A3    | Builds DAG; updates courses.knowledgeMap        |
| onUserCreate                  | A6    | Merges anonymous cookie progress                |
| sessionEventCallback          | A1–A2 | Ingests proxy telemetry into gemini_sessions    |

## A.5 SDK Client Events

The TypeScript SDK surfaces a small event union to the UI layer. Table A.5 documents events consumers should handle.

Table A.5: @sprout/gemini-proxy-sdk client event catalogue.

| Event         | Payload / handling notes                      |
|---------------|-----------------------------------------------|
| connected     | Upstream ready; enable microphone/camera      |
| message       | Model or user transcript chunk                |
| usage         | {inputTokens, outputTokens} for meter UI      |
| function_call | Routed to backend callback (future tools)     |
| disconnected  | Includes code and reason; map 44xx to UX copy |
| reconnecting  | Exponential backoff in progress               |

# References

- [1] MidScene.js Project. *MidScene.js: AI-Powered Browser Automation SDK*. 2024. url: <https://midscenejs.com/>.
- [2] Sprout Engineering Team. *MidScene.js Multi-Agent Research and Implementation Patterns*. Internal Technical Report. 2026.
- [3] A. C. Graesser et al. “AutoTutor: An Intelligent Tutoring System with Mixed-Initiative Dialogue”. In: *IEEE Transactions on Education* 48.4 (2005), pp. 612–618. doi: 10.1109/TE.2005.856149.
- [4] Google DeepMind. *Gemini: A Family of Highly Capable Multimodal Models*. Technical Report. 2024. url: <https://deepmind.google/technologies/gemini/>.
- [5] A. Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *Proceedings of ICML*. Vol. 139. 2021, pp. 8748–8763.
- [6] LiveKit Inc. *LiveKit Documentation: Real-Time Voice and Video Infrastructure*. 2024. url: <https://docs.livekit.io/>.
- [7] I. Fette and A. Melnikov. *The WebSocket Protocol*. RFC 6455, IETF. 2011. doi: 10.17487/RFC6455.
- [8] M. Jones, J. Bradley, and N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, IETF. 2015. doi: 10.17487/RFC7519.
- [9] T. Schick, J. Dwivedi-Yu, R. Dessì, et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.
- [10] Sprout Engineering Team. *Function Calling Integration Analysis and Opportunities*. Internal Technical Report. 2026.
- [11] P. Chen et al. “KnowEdu: A System to Construct Knowledge Graph for Education”. In: *IEEE Access* 6 (2018), pp. 31553–31563. doi: 10.1109/ACCESS.2018.2839607.

- [12] Y. Hu, Y. Koren, and C. Volinsky. “Collaborative Filtering for Implicit Feedback Datasets”. In: *Proceedings of ICDM*. 2008, pp. 263–272. doi: 10.1109/ICDM.2008.22.
- [13] H. Steck. “Calibrated Recommendations”. In: *Proceedings of RecSys*. 2018, pp. 154–162. doi: 10.1145/3240323.3240372.
- [14] Y. A. Malkov and D. A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2018). doi: 10.1109/TPAMI.2018.2889473.
- [15] N. N. Anh. *Balancing Relevancy and Diversity with Two-Stage Maximal Marginal Relevance (2-Stage MMR) for Fashion Recommendation*. Internship 1 Final Report, Ho Chi Minh City University of Technology. 2026.
- [16] J. G. Carbonell and J. Goldstein. “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries”. In: *Proceedings of SIGIR*. 1998, pp. 335–336. doi: 10.1145/290941.291025.
- [17] Google. *Firebase Documentation: Authentication, Firestore, and Cloud Functions*. 2024. url: <https://firebase.google.com/docs>.
- [18] Google. *Cloud Run: Managed Container Platform Documentation*. 2024. url: <https://cloud.google.com/run/docs>.
- [19] A. Radford et al. “Robust Speech Recognition via Large-Scale Weak Supervision”. In: *arXiv preprint arXiv:2212.04356* (2022).